
AWS Transit Gateway – Lab with Notes & Task

Overview

An **AWS Transit Gateway (TGW)** acts like a **central cloud router** that allows you to **connect multiple VPCs and on-premises networks** via a **hub-and-spoke model**, replacing the complexity of many-to-many VPC peering relationships.

Why Use Transit Gateway?

Benefit	Description
Simplified Network	Centralized hub to reduce VPC peering complexity
Scalability	Supports thousands of VPC connections
Security	Traffic stays within AWS, encrypted and off the public internet
Supports Hybrid	Enables on-premises to AWS connectivity using VPN or Direct Connect
Multicast Support	Supports multicast for specific use cases like video streaming

VPC Peering vs Transit Gateway

Feature	VPC Peering	Transit Gateway
Transitive Routing	 Not supported	 Supported
Number of Connections	Limited & complex	Centralized, scalable
On-premises Support	 Not supported	 Supported via VPN/DX
Maintenance Complexity	High	Lower due to centralized architecture



Transit Gateway Limitations (Default Limits)

- **20** route tables per Transit Gateway
- **10,000** routes per route table
- **50** attachments per Transit Gateway
- **5** Transit Gateways per VPC



Transit Gateway Architecture Components

Component	Description
Transit Gateway	Central router between VPCs and other networks
Attachment	Connection between a VPC and the Transit Gateway
Transit Route Table	Controls routing between attachments
Propagation	Allows VPC routes to be shared with the TGW route table automatically



Lab Task – Setup VPC Peering via Transit Gateway



Prerequisites

- AWS Console access
- IAM permissions to create VPCs, EC2, Transit Gateway



Lab Steps

- ◆ **Step 1: Create First VPC (VPC-A)**
 - Name: `VPC-A`
 - CIDR block: `10.10.0.0/16`
 - ◆ **Step 2: Create a Public Subnet in VPC-A**
 - CIDR block: `10.10.1.0/24`
 - Availability Zone: `us-east-1a`
 - ◆ **Step 3: Create and Attach an Internet Gateway**
 - Name: `igw-vpc-a`
 - Attach it to `VPC-A`
 - ◆ **Step 4: Create a Route Table in VPC-A**
 - Name: `public-rt-vpc-a`
 - Associate it with the public subnet
 - Add route:
`0.0.0.0/0 → igw-vpc-a`
 - ◆ **Step 5: Launch EC2 in Public Subnet of VPC-A**
 - AMI: Amazon Linux 2
 - Instance Type: `t2.micro`
 - Security Group: Allow SSH (port 22)
 - Name: `ec2-vpc-a`
-

♦ **Step 6: Create Second VPC (VPC-B)**

- Name: `VPC-B`
- CIDR block: `10.20.0.0/16`

♦ **Step 7: Create Private Subnet in VPC-B**

- CIDR block: `10.20.1.0/24`

♦ **Step 8: Launch EC2 in Private Subnet of VPC-B**

- Name: `ec2-vpc-b`
- AMI: Amazon Linux 2
- Make sure you add the instance in a security group that allows SSH from VPC-A's CIDR

♦ **Step 9: Create Transit Gateway**

- Name: `my-transit-gateway`
- Leave default ASN and settings

♦ **Step 10: Create Transit Gateway Attachments**

1. **Attachment A:**

- Attach `VPC-A` to the Transit Gateway
- Select subnet from `10.10.1.0/24`

2. **Attachment B:**

- Attach `VPC-B` to the Transit Gateway

- Select subnet from 10.20.1.0/24

◆ Step 11: Update Route Tables

- In VPC-A Public Route Table:
 - Add route: 10.20.0.0/16 → Transit Gateway
- In VPC-B Route Table (main/private):
 - Add route: 10.10.0.0/16 → Transit Gateway

✓ Step 12: Test VPC-to-VPC Communication

- SSH into the EC2 instance in VPC-A (public subnet)
- From there, ping the private IP of the EC2 instance in VPC-B
- If successful, Transit Gateway setup is working!

🧹 Step 13: Clean-Up (IMPORTANT)

- Terminate EC2 instances
- Detach & delete Transit Gateway attachments
- Delete Transit Gateway
- Delete subnets, route tables, IGW, VPCs

🔍 Use Cases for Transit Gateway

- Hybrid connectivity (AWS + on-prem via VPN/DX)
 - Centralized egress for VPCs
 - Multi-region app delivery
 - Microservices in separate VPCs with centralized routing
 - Mergers/acquisitions needing quick inter-VPC setup
-

Learning Outcome

By completing this lab, you'll understand:

- How Transit Gateway simplifies network peering
 - How to route traffic between VPCs securely without using public internet
 - How to validate and test inter-VPC traffic
-